PlantHealthMonitor

Manual

Casey Stijlaart

Table of Contents

1. Introduction

PlantHealthMonitor (PHM) is an intelligent plant monitoring and care system that combines embedded IoT hardware with a cross-platform mobile application. The system collects environmental sensor data from a plant's surroundings, classifies plant health in real time using on-device machine learning, and pushes actionable care recommendations to your phone or desktop.

This manual covers everything you need to set up the hardware, install the app, configure your plants, and understand the information displayed on screen.

1.1. Key Features

- **Real-time sensor monitoring** — soil moisture, temperature, humidity, and light intensity.
- **On-device ML classification** — a trained neural network (MLP) running on the ESP32 classifies plant health as healthy, moderate, or high risk without a cloud round-trip.
- **ML-powered time predictions** — a second regression model predicts how many minutes until a care action (e.g. watering) becomes urgent.
- **Rule-based care actions** — the system applies sensor thresholds to decide which actions to recommend (water, reduce temperature, increase light, increase humidity).
- **Push notifications** — instant alerts when a plant needs attention, plus an optional daily summary.
- **Multi-plant support** — manage several plants, each with its own device and preference profile.
- **Cross-platform app** — Android, iOS, and Windows are supported. Device pairing requires the Android or Windows app; the iOS app supports monitoring and management only (see Platform Differences).
- **App-based device setup** — new devices are paired over Bluetooth (BLE); no credentials are compiled into the firmware.

2. System Overview

PlantHealthMonitor consists of three layers that work together:

Layer	Component	Responsibility
Edge	ESP32 microcontroller + sensors	Collects sensor readings, runs TinyML inference, syncs data to the cloud every 3 hours.
Cloud	Supabase (PostgreSQL + Realtime)	Stores sensor readings and plant settings; delivers live updates to connected apps.
App	Flutter (Android / iOS / Windows)	Displays live readings, charts, and recommendations; sends push notifications; stores your preferences.

2.1. Data Flow

```
Sensors (soil, temp, humidity, light)
    
ESP32 — preprocessing → TinyML inference → risk class + recommendations
    
Supabase cloud (plant_readings table)
    
Flutter app (realtime subscription) → dashboard + notifications
```

2.2. Health Risk Levels

The system classifies each reading into one of three risk levels:

Level	Label	Meaning
0	Healthy	All sensor values fall within your plant's preferred bands. No action needed.
1	Moderate Risk	One or more values are outside the preferred band but not critical. Monitor closely.
2	High Risk	Critical conditions detected. Immediate action is recommended.

3. Hardware Setup

3.1. Bill of Materials

Qty	Item	Notes
1	ESP32 development board	Any standard 38-pin variant
1	Capacitive soil moisture sensor	Analog output, connects to GPIO 34
1	DHT22 temperature & humidity sensor	Connects to GPIO 4
1	LDR (light-dependent resistor)	Analog output with 10 kΩ pull-down, connects to GPIO 35
1	10 kΩ resistor	For the LDR voltage divider
—	Jumper wires	Male-to-male and male-to-female
1	USB-A to micro-USB or USB-C cable	For flashing and serial monitor

3.2. Wiring Diagram

Sensor pin	ESP32 pin
Soil moisture — AOUT	GPIO 34
DHT22 — DATA	GPIO 4
DHT22 — VCC	3.3 V
DHT22 — GND	GND
LDR — one leg (with 10 kΩ to GND)	GPIO 35
LDR — other leg	3.3 V

NOTE GPIO 34 and 35 are input-only pins on the ESP32. Do not connect any output signal to them.

3.3. Flashing the Firmware

The firmware is managed with PlatformIO. No credentials or plant configuration need to be entered at build time — all of that is configured later from the app during pairing.

1. Install PlatformIO (VS Code extension or CLI).
2. Navigate to the firmware source:

   ```
   cd src/firmware
   ```

3. Build and upload to the device:

   ```
   platformio run -e esp32_provisioning -t upload
   ```

4. Verify the device is running correctly:

   ```
   platformio device monitor -b 115200
   ```

   On first boot the device advertises itself over Bluetooth as `PHM-XXXXXX` and waits to be paired from the app.

> **TIP** If you have multiple plants, flash each ESP32 with the same firmware — each device generates its own unique identifier and is configured individually during pairing.

4. Mobile App Setup

4.1. Platform Differences

The app behaves differently per platform. This matters mostly for the initial device setup:

Platform	Device pairing	Notes
Android / Windows	Full support	BLE device scanning and WiFi configuration are available. Use one of these platforms to pair new devices.
iOS	Not available	The sideloaded iOS build (free Apple ID) cannot use the BLE device scanning and WiFi network scanning capabilities required for pairing. Once a device has been paired from Android or Windows, the iOS app offers the full monitoring experience: dashboard, charts, preferences, notifications, and device commands all work, since they run through the cloud.

4.2. Installation

All app releases are published on the PlantHealthMonitoring GitHub releases page.

4.2.1. Android / Windows

- **Android**: download and install the APK (`phm_app_<version>.apk`) from the releases page (or build it from source, see below).
- **Windows**: download the release archive (`phm_app_windows_<version>.zip`) from the releases page, extract it, and run `phm_app.exe`.

4.2.2. iOS

Download `phm_app_<version>.ipa` from the releases page. Full sideloading instructions using Sideloadly on Windows are included with the release.

NOTE Within the EU, sideloaded apps are not time-limited. Outside the EU, sideloading with a free Apple ID expires after 7 days, after which the app must be re-signed. Device pairing is not available on iOS — pair your devices from the Android or Windows app first.

4.3. Building from Source

```
cd src/app
flutter pub get
```

```
flutter run                 # connected device or emulator
flutter run -d windows      # Windows desktop
flutter build apk           # Android release APK
```

Flutter 3.x or later is required.

4.4. Pairing a Device (Android / Windows)

New devices are set up entirely from the app — no firmware changes are needed.

1. Power on the freshly flashed ESP32. It advertises itself over Bluetooth as `PHM-XXXXXX`.
2. In the app, choose **Add Device**. The app scans for nearby unpaired devices.
3. Select your device and enter your WiFi network (2.4 GHz) and password.
4. The app transfers the configuration over BLE. The device connects to WiFi and registers itself in the cloud.
5. When the device appears in the app, enter the plant name and care preferences to finish the setup. The device picks up the plant assignment automatically and runs its first measurement.

To remove a device, use **Remove Device** in the app: the device erases its history, credentials, and cloud registration, and reboots into pairing mode so it can be set up again.

5. Using the App

5.1. Dashboard

The dashboard is the main screen. It shows a card for each plant found in the cloud, including:

- **Status badge** — Healthy / Moderate Risk / High Risk.
- **Last updated** — how long ago the most recent reading arrived.
- **Sensor tiles** — four tiles showing the current values for soil moisture, temperature, humidity, and light.
- **Recommendation summary** — a short text describing any care action needed and the estimated time until it is required.

Tap a sensor tile to see which preference band the current reading falls into (low / mid / high).

Pull down on the dashboard to force a refresh from the cloud.

5.2. Plant Selector

If you have more than one plant, use the dropdown at the top of the dashboard to switch between them. Each plant is identified by the plant name entered during setup.

5.3. Sensor Detail

Tapping the **Data** button on a plant card opens the sensor history charts. Charts show the last week of readings (up to 56 data points at 3-hour intervals) for each sensor.

5.4. Plant Preferences

Tap the **Preferences** button (or navigate via the menu) to edit the care preferences for a plant.

Each of the four sensors has a preference selector with three options:

Option	Meaning
Low	The plant prefers lower values for this sensor (e.g. a cactus for soil moisture).
Mid	The plant prefers moderate values.
High	The plant prefers higher values (e.g. a tropical plant for humidity).

After editing, tap **Save**. The preferences are synced to the cloud and used by the recommendation engine on the next sensor reading.

5.5. Notifications

5.5.1. Instant Alerts

The app sends a push notification any time a plant's risk class changes to Moderate or High Risk. Duplicate notifications are suppressed — you will not receive the same alert twice for the same condition.

5.5.2. Daily Summary

You can enable a daily summary notification that reports the status of all your plants at a chosen time. To configure this:

1. Open **Settings** from the navigation menu.
2. Toggle **Daily Report** on.
3. Pick a time using the time picker.

The app schedules this notification locally on your device and it fires even if the app is in the background.

5.5.3. Disabling Notifications

Toggle **Instant Alerts** or **Daily Report** off individually in **Settings**.

6. Understanding Readings & Recommendations

6.1. Sensor Ranges

The sensors report percentages (0–100 %) except temperature, which is in degrees Celsius.

Sensor	Low band	Mid band	High band
Soil moisture (%)	0–33	34–66	67–100
Temperature (°C)	< 18	18–28	> 28
Humidity (%)	0–40	41–70	71–100
Light (%)	0–33	34–66	67–100

6.2. Recommendation Actions

The recommendation engine can trigger four care actions:

Action	What it means
Water the plant	Soil moisture is below the preferred band or declining towards it. The time estimate shows how long until watering becomes urgent.
Reduce temperature	Temperature is above the preferred band.
Increase light	Light level is below the preferred band.
Increase humidity	Humidity is below the preferred band.

The timing estimate (e.g. "water in 4 hours") is produced by a trained regression neural network running on the ESP32 — it predicts how many minutes until the action becomes urgent based on sensor history. The action flags themselves (water / reduce temp / etc.) are determined by rule-based threshold checks on the raw sensor values, combined with the ML risk class.

When the risk class is 0 (Healthy), no actions are triggered and the recommendation summary shows a positive status message.

7. Troubleshooting

7.1. ESP32 is not connecting to WiFi

- The ESP32 only supports 2.4 GHz networks. Make sure you did not enter a 5 GHz network during pairing.
- If the credentials were entered incorrectly, remove the device in the app (or wait — a failed first connection automatically resets it to pairing mode) and pair it again.
- Open the serial monitor (`platformio device monitor -b 115200`) to read detailed connection logs.

7.2. No data appears in the app

- Confirm the ESP32 is powered on and has successfully connected to WiFi (check the serial monitor).
- The device measures every 3 hours. Use the measure-now command in the app to request an immediate reading, which the device picks up within a few seconds.
- Check that a plant has been assigned to the device — an unassigned device does not upload readings.

7.3. App shows stale data

- Pull down on the dashboard to force a cloud refresh.
- Check your internet connection on the mobile device.

7.4. Notification not received

- Make sure notifications are enabled for the app in your device's system settings.
- On Android 13 and later, the app requires the `POST_NOTIFICATIONS` runtime permission. Grant it via **Settings** → **Apps** → **PlantHealthMonitor** → **Permissions**.

7.5. Preferences are not saving

- The app requires an internet connection to sync preferences to the cloud. Connect to the internet and try again.
- If the problem persists, force-quit the app and re-enter your preferences.

8. Technical Reference

8.1. Cloud Database Schema

8.1.1. plant_readings

Column	Type	Description
plant_label	text	Unique identifier for the plant (matches firmware `PLANT_LABEL`).
device_id	text	Hardware identifier of the ESP32.
device_name	text	Human-readable device name.
soil_moisture_pct	float	Soil moisture as a percentage (0–100).
temperature_c	float	Ambient temperature in °C.
humidity_pct	float	Relative humidity as a percentage (0–100).
light_level_pct	float	Light intensity as a percentage (0–100).
risk_class	int	Health classification: 0 = healthy, 1 = moderate, 2 = high.
action_water	bool	Whether watering is recommended.
action_reduce_temp	bool	Whether reducing temperature is recommended.
action_increase_light	bool	Whether increasing light is recommended.
action_increase_humidity	bool	Whether increasing humidity is recommended.
recommendation_summary	text	Human-readable recommendation including time predictions.
timestamp	timestamptz	Reading time in UTC.

8.1.2. plant_settings

Column	Type	Description
plant_label	text	Unique plant identifier (primary key).
soil_preference	text	`low`, `mid`, or `high`.
temperature_preference	text	`low`, `mid`, or `high`.
humidity_preference	text	`low`, `mid`, or `high`.
light_preference	text	`low`, `mid`, or `high`.
version	int	Incremented on each save; used for synchronisation.

8.2. Device Configuration Reference

All device configuration is provided at runtime during BLE pairing and stored on the device:

Setting	Provided via	Stored in
WiFi SSID + password	App during pairing	LittleFS pairing document (`/pairing_state.txt`).
Cloud API key + URL	App during pairing	LittleFS pairing document.
Device ID	Generated by the device on first boot	Non-volatile storage (NVS).
Device name	Derived from the hardware MAC address	—
Plant name + preferences	App after pairing	Cloud (`plant_settings`); fetched by the device before each cycle.

Removing a device marks it unpaired, which erases the stored WiFi credentials by construction.

8.3. ML Models

The firmware contains two trained neural networks that run directly on the ESP32:

Health risk classifier — a multi-layer perceptron (MLP) with a hidden layer (ReLU activation) and a softmax output layer. It takes normalised sensor features as input and outputs a risk class (0, 1, or 2) with per-class confidence scores. A rule-based fallback classifier is also available and can be selected via the pluggable `MLLayer` backend. Edge Impulse models are additionally supported.

Predictive irrigation model — a regression MLP with a hidden layer (ReLU) and a linear output. It was trained on log-transformed target values (minutes until watering is needed) and uses `expm1` to convert the output back to minutes. This value drives the time estimate in the recommendation summary.

The care action flags (water / reduce temp / increase light / increase humidity) are generated by a separate rule-based `RecommendationEngine` that compares raw sensor readings against the per-plant preference thresholds, using the ML risk class as an additional input.

9. Future Possibilities

This section outlines improvements and extensions that could be added to PlantHealthMonitor in future iterations.

9.1. BLE Data Sync and iOS Pairing

BLE provisioning is implemented: devices are configured from the app over Bluetooth, with no credentials in the firmware. Two related extensions remain open:

- **BLE data sync** — the BLE channel could also serve as a local data path for users who do not want to connect the device to the internet at all, syncing readings directly to the phone when in range.
- **iOS pairing support** — with a paid Apple Developer account, the BLE and WiFi scanning capabilities required for pairing could be enabled in the iOS build, removing the current dependency on Android or Windows for device setup.

9.2. Built-in Plant Profiles

At the moment, users set preferences manually (low / mid / high per sensor). A future version could ship a plant species library with pre-defined care profiles. Users would pick their plant from a list (e.g. "Snake Plant", "Peace Lily", "Basil") and the correct preferences would be populated automatically. The library could be:

- Bundled with the app for offline use.
- Pulled from a community-maintained cloud database so new species can be added without an app update.

User-defined custom profiles would remain available for unlisted species.

9.3. Support for Additional Sensor Types

The current sensor set covers the most common care factors, but the hardware layer could be extended to support:

- **pH sensor** — soil acidity monitoring, relevant for sensitive species.
- **EC (electrical conductivity) sensor** — measures nutrient levels in the soil.
- **CO□ sensor** — useful for indoor grow environments.

The pluggable `MLLayer` architecture already makes it relatively straightforward to retrain models with additional input features once new sensors are added.

9.4. Improved ML Model Training Pipeline

The current ML models are trained offline and their weights are exported as C++ header files. A

more scalable approach would be to:

- Continuously collect labelled sensor data from all deployed devices (with user consent).
- Retrain the models periodically in the cloud.
- Push updated weights to devices via an over-the-air (OTA) update mechanism, which the current firmware does not yet include.

This would allow the models to improve over time and adapt to a wider variety of plants and environments.

9.5. Multi-User and Sharing Support

Currently each installation is tied to a single Supabase project. Adding an authentication layer would allow:

- Multiple users to share ownership of the same plant (e.g. housemates).
- Read-only guest access so a plant-sitter can monitor status without changing settings.
- Per-user notification preferences independent of device configuration.

10. Appendix : Glossary

Term	Definition
ESP32	Low-cost Wi-Fi + Bluetooth microcontroller used as the sensing device.
TinyML / TFLM	TensorFlow Lite Micro — a framework for running ML models on microcontrollers.
Plant label	A string identifier that links a hardware device to its cloud records and app preferences.
Risk class	An integer (0, 1, or 2) output from the ML model indicating the current health level.
Preference band	The user-selected range (low / mid / high) for a sensor that the recommendation engine uses to evaluate readings.
Supabase	The managed cloud backend providing PostgreSQL storage and realtime change events.
OTA	Over-the-Air firmware update — flashing new firmware over WiFi without a USB cable. Not supported by the current firmware; listed as a future improvement.